TECHNICAL INTERVIEW PREP

# Senior React Developer — Technical Q&A

A focused knowledge base of the technical questions most likely to come up, with answers built to hold up under follow-up questioning. Ordered from most-probable to niche.

> **Stack in focus:** React ecosystem & internals · Modern JavaScript/TypeScript · Jest + React Testing Library · Scalable frontend architecture & shared component libraries · SPFx / SharePoint Online integration.

CONTENTS



## 1 React Internals & Rendering

*The interviewer wants to see you understand *why* React behaves the way it does, not just the API surface.*

### Q What actually happens when a component re-renders?

"Render" means React **calls your component function** to produce a new element tree (the virtual DOM). React then **reconciles** that tree against the previous one, computes the minimal set of DOM mutations, and commits them. Rendering does not necessarily touch the DOM — a render that produces identical output commits nothing.

The mental model to state: *render phase* (pure, may be paused/restarted, no side effects) vs *commit phase* (DOM is mutated, refs assigned, layout effects run synchronously, passive effects scheduled).

> **Follow-up trap:** "Does a state change always re-render?" — No. If you set state to the same primitive value ( `Object.is` equality), React bails out of the re-render for that component.

### Q Explain reconciliation and the diffing heuristics.

React's diff is O(n) because of two assumptions: (1) elements of **different types produce different trees** — a change from `<div>` to `<span>` tears down the subtree and rebuilds it; (2) **keys** tell React which children are stable across renders. Same type + same position → React reuses the DOM node and only updates changed props.

### Q Why are keys important, and why is index-as-key a problem?

Keys give each list item a stable identity so React can match old and new children. With **index keys**, inserting or reordering shifts every index, so React thinks items changed in place — it reuses

the wrong DOM nodes. Symptoms: wrong input values sticking to the wrong row, lost focus, broken animations, stale uncontrolled state.

Use a stable domain ID. Index keys are only safe for a static, append-only, never-reordered list with no local state.

**Q What is the Fiber architecture and why does it exist?**

Fiber is the reimplementation of React's core (since v16) that makes rendering **interruptible**. Each element has a corresponding fiber node holding its state, and work is split into units so React can pause, prioritise, and resume — enabling concurrent features. Before Fiber, reconciliation was a synchronous recursive stack that couldn't be interrupted, so a large update could block the main thread.

**Q Why does StrictMode double-invoke my components in development?**

To surface **impure renders and unsafe effects**. React intentionally double-invokes render functions and effect setup/cleanup (dev only) so that side effects hidden in render, or missing effect cleanup, become visible. If double-invocation breaks your component, that's a real bug — usually a mutation during render or an effect that isn't idempotent.

> **Say this:** It's dev-only and does not run in production, but it's a preview of concurrent-rendering behaviour where components may render without committing.

**Q Controlled vs uncontrolled components — when would you choose each?**

**Controlled:** value lives in React state, single source of truth, needed for validation-on-change, conditional formatting, or interdependent fields. **Uncontrolled:** the DOM holds the value, read via a ref — lighter, good for simple forms, file inputs, or integrating non-React widgets. In enterprise forms you'll often go controlled via a form library (React Hook Form) that keeps most inputs uncontrolled for performance while still validating.

## 2 Hooks in Depth

*Expect closure and dependency-array questions — this is where seniority shows.*

**Q What are the Rules of Hooks and what breaks if you ignore them?**

Call hooks **only at the top level** (never in conditions, loops, or nested functions) and **only from React functions**. React tracks hook state by call order, not by name — a conditional hook shifts the order between renders, so React associates state with the wrong hook and everything corrupts. That's the whole reason the linter rule exists.

**Q Explain the stale closure problem with useEffect.**

Each render creates fresh function instances that "close over" that render's props/state. If an effect captures a value but that value isn't in the dependency array, the effect keeps seeing the **old captured value** — the classic stale closure. Example: a `setInterval` that reads `count` but has `[]` deps will forever log the initial count.

Fixes: add the value to deps, or use the **functional updater** `setCount(c => c + 1)` so you don't depend on the captured value, or hold it in a ref.

> **Follow-up trap:** "Why not just remove the dependency array?" — Then the effect runs every render, which usually re-subscribes/re-fetches on every keystroke. Deps aren't a suggestion; they define correctness.

**Q useMemo vs useCallback — what's the real difference and when do they matter?**

`useMemo` memoises a **computed value**; `useCallback` memoises a **function reference**. `useCallback(fn, deps)` is literally `useMemo(() => fn, deps)`.

They matter in two cases: (1) an expensive calculation you don't want to redo each render; (2) a value/function passed to a `React.memo` child or used as another hook's dependency, where a new reference each render would defeat memoisation or retrigger effects. Wrapping trivial values is net-negative — the cache bookkeeping costs more than it saves.

> **Senior signal:** Mention that React Compiler (React 19+) auto-memoises, reducing the need for manual `useMemo` / `useCallback` — but you still need to understand the mechanics.

### Q useEffect vs useLayoutEffect?

`useLayoutEffect` fires **synchronously after DOM mutations but before the browser paints** — use it when you must read layout (measure a node) and mutate before the user sees a flicker. `useEffect` fires **after paint**, asynchronously — the default for data fetching, subscriptions, logging. Overusing layout effects blocks paint and hurts performance.

### Q When do you reach for useRef?

Two uses: (1) accessing a DOM node imperatively (focus, scroll, measure, integrate a third-party lib); (2) holding a **mutable value that persists across renders without triggering a re-render** — timers, previous values, "latest" callbacks. Key point: mutating `ref.current` does not cause a render, which is exactly why it's not for anything the UI displays.

### Q How do you design a good custom hook?

A custom hook is just a function starting with `use` that composes other hooks. Good ones have a **clear single responsibility** (e.g. `useDebounce`, `useLocalStorage`, `usePagination`), return a stable, well-typed interface, and encapsulate the effect/cleanup so callers can't misuse it. They extract *logic*, not markup — that's the distinction from a component.

## 3 Performance & Optimisation

*Enterprise React apps get slow; they'll want your systematic approach, not random micro-tweaks.*

### Q A page re-renders too much. How do you diagnose it?

State the method, in order:

- **Measure first** — React DevTools Profiler to see which components render and why ("what caused this render"). Don't optimise on a hunch.
- Identify the trigger: parent state change cascading down, a new object/array/function prop breaking memoisation, or context value changing.
- Apply the right fix: `React.memo` on pure children, stabilise props with `useMemo` / `useCallback`, lift or colocate state, split context.
- Re-measure to confirm.

### Q What does React.memo do and what are its limits?

It memoises a component so it skips re-rendering when props are **shallow-equal** to the previous render. Limits: it only helps if props are actually stable — pass a fresh inline object/array/callback and it re-renders anyway. It also doesn't stop re-renders caused by `useState` / `useContext` *inside* the component. It's a targeted tool, not a blanket wrapper.

> **Follow-up trap:** "Should you wrap every component in memo?" — No. The shallow-compare has a cost, and most components are cheap to render. Profile, then memo the proven-expensive ones.

**Q How do you keep a large bundle fast?**

- **Code splitting** with `React.lazy` + `Suspense`, or route-based splitting, so users download only what a route needs.
- **Tree shaking** — import named members, avoid barrel files that pull in whole libraries, watch for side-effectful modules.
- Analyse with a bundle analyzer; replace heavy deps (e.g. moment → date-fns/day.js).
- For long lists, **virtualise** (react-window / react-virtuoso) so only visible rows render.

**Q Explain the "lift state up vs colocate state" trade-off.**

State should live at the **lowest common ancestor** that needs it. Lifting too high makes an unrelated subtree re-render on every change (a common cause of app-wide sluggishness). Colocating state near where it's used shrinks the render blast radius. The senior instinct: push state *down* until something forces it up.

## 4 State Management & Data Fetching

*Have an opinion, and be able to justify it against alternatives.*

**Q Context API vs a state library (Redux/Zustand) — how do you choose?**

**Context** is a dependency-injection mechanism, not a state manager — great for low-frequency global values (theme, locale, current user). Its weakness: every consumer re-renders when the value changes, so high-frequency updates are a footgun.

Reach for **Redux Toolkit** when you need predictable global state, middleware, time-travel/devtools, and team conventions at scale; **Zustand/Jotai** for lighter, hook-first stores with selective subscriptions. State the decision rule: complexity and update frequency drive the choice, not fashion.

**Q Why separate "server state" from "client state"?**

Because server data is **asynchronous, shared, and can go stale** — it needs caching, background refetching, deduping, and invalidation, which general state managers don't give you. Tools like **TanStack Query** or RTK Query own that lifecycle (loading/error/stale states, cache keys, retries), leaving client state (UI toggles, form state) to `useState`/context. Conflating the two is why so many apps hand-roll buggy fetch-and-cache logic.

> **Relevant to this role:** In a SharePoint/M365 context you're often consuming Graph and SharePoint REST endpoints — a query cache dramatically simplifies that.

**Q How do you handle race conditions in data fetching with hooks?**

If a component fires a request, the deps change, and it fires another, responses can resolve out of order and the stale one wins. Guard it with an **AbortController** or an *ignore flag* in the effect cleanup:

```
useEffect(() => {
  let ignore = false;
  fetchData(id).then(res => { if (!ignore) setData(res); });
  return () => { ignore = true; }; // cleanup on id change/unmount
}, [id]);
```

Or let a data-fetching library handle cancellation for you.

## 5 Modern JavaScript & TypeScript

*Deep JS knowledge underpins everything; they may test fundamentals directly.*

**Q** **Explain closures with a practical use.**

A closure is a function bundled with the lexical scope it was defined in, so it retains access to those variables after the outer function returns. Practical uses: **data privacy** (module pattern, private state), **function factories** (a configured logger), **memoisation caches**, and event handlers/callbacks that "remember" values. In React, every render's handlers are closures over that render's state — which is exactly the source of stale-closure bugs.

**Q** **Event loop: explain microtasks vs macrotasks.**

JS is single-threaded with an event loop. The **call stack** runs sync code; async callbacks queue up. **Microtasks** (Promise `.then`, `queueMicrotask`, `await` continuations) drain *completely* after the current task and *before* the next macrotask. **Macrotasks** (`setTimeout`, I/O, UI events) run one per loop iteration. So a resolved Promise callback always fires before a `setTimeout(0)` queued at the same time.

**Q** **== vs === , and how does JS coerce types?**

`===` compares value *and* type with no coercion. `==` applies the abstract equality algorithm, coercing operands (e.g. `'' == 0` is `true`, `null == undefined` is `true`). Rule: use `===` by default; the only common defensible `==` is `x == null` to catch both `null` and `undefined`.

**Q** **How do you avoid mutating state, and why does it matter in React?**

React detects change by reference (`Object.is`). Mutating an object/array in place keeps the same reference, so React (and `memo`) sees "no change" and skips the update — or worse, updates unpredictably. Produce **new references**: spread (`{...obj, x}`, `[...arr, item]`), `map` / `filter`, or a library like **Immer** for deep updates. This is also why Redux Toolkit uses Immer under the hood.

**Q** **TypeScript: generics, and when do you use them?**

Generics let you write reusable, type-safe code parameterised over a type — the type flows through instead of being fixed or `any`. Classic example: a `useFetch<T>()` hook whose return is typed to the caller's shape, or a `List<T>` component. Mention practical tools: **union & discriminated-union types** for state machines, **utility types** (`Partial`, `Pick`, `Omit`, `Record`), and preferring `unknown` over `any` at boundaries.

> **Follow-up trap:** "any vs unknown?" — `any` disables checking and spreads silently; `unknown` is type-safe — you must narrow it before use. Use `unknown` for untrusted input (API responses).

**Q** **Explain async/await vs Promises, and error handling.**

`async/await` is syntactic sugar over Promises that reads sequentially. Errors: wrap `await` in `try/catch`; for parallelism use `Promise.all` (fails fast) or `Promise.allSettled` (collects all results/errors). A common senior point: don't `await` in a loop when calls are independent — fire them together and `await Promise.all` to avoid serialising latency.

## 6 Testing — Jest & React Testing Library

*The listing calls this "essential." Expect concrete, hands-on questions here.*

**Q** **What's the core philosophy of React Testing Library?**

**Test behaviour, not implementation.** Query and assert the way a user experiences the UI — by role, label, and text — not by component internals, state, or CSS classes. The payoff: tests survive

refactors (you can rewrite the internals and tests still pass as long as behaviour holds). The guiding line: "the more your tests resemble the way your software is used, the more confidence they give you."

**Q Which queries do you use, and in what priority?**

Priority order RTL recommends:

- **getByRole** (with `name`) — closest to how users and assistive tech navigate; the default choice.
- **getByLabelText** — form fields.
- **getByPlaceholderText**, **getByText** — for non-interactive content.
- **getByTestId** — last resort when nothing semantic fits.

And the variants: `getBy*` throws if absent, `queryBy*` returns null (for asserting absence), `findBy*` returns a Promise (for async appearance).

**Q How do you test something asynchronous — data appearing after a fetch?**

Use `findBy*` or `waitFor`, which retry until the assertion passes or times out:

```
test('shows users after load', async () => {
  render(<UserList />);
  // findBy waits for the element to appear
  expect(await screen.findByText('Ada Lovelace')).toBeInTheDocument();
});
```

Never use arbitrary `setTimeout` in tests. Wrap user interactions with `userEvent` (which is async) and `await` them.

**Q userEvent vs fireEvent — which and why?**

Prefer **@testing-library/user-event**. `fireEvent` dispatches a single raw DOM event; `userEvent` simulates the *full* interaction (a click also fires pointerdown/focus/mouseup, typing fires keydown/keypress/input per character). It's far closer to real user behaviour and catches bugs `fireEvent` misses.

**Q How do you mock modules, timers, and network calls in Jest?**

- **Modules:** `jest.mock('./api')`, or `jest.spyOn(obj, 'method')` to observe/stub a single function.
- **Timers:** `jest.useFakeTimers()` + `jest.advanceTimersByTime()` to test debounces/intervals deterministically.
- **Network:** best practice is **MSW (Mock Service Worker)** — intercept at the network layer so components run real fetch logic against mocked responses, instead of mocking `fetch` itself.

> **Senior signal:** Say you avoid over-mocking. Mocking everything tests your mocks, not your app. Mock the boundary (network, time), keep the rest real.

**Q Unit vs integration vs snapshot tests — how do you balance them?**

Favour **integration-style** tests that render a component with its children and assert user-visible behaviour — best confidence per test. Pure **unit** tests for complex logic/utilities/custom hooks (`renderHook`). **Snapshot** tests sparingly — large snapshots rot, get blindly updated, and assert nothing meaningful; small, intentional snapshots for stable output only. Chase meaningful coverage of critical paths, not a 100% number.

## 7 Scalable Architecture & Component Libraries

*The role explicitly mentions shared component libraries and scalable architecture — have opinions ready.*

**Q How do you structure a large React codebase?**

Move from type-based folders (`/components`, `/hooks`) to **feature-based / domain folders** that colocate a feature's components, hooks, tests, and state. Keep a shared `/ui` or design-system layer for primitives. Enforce **dependency direction**: features depend on shared, never the reverse; avoid cross-feature imports. Mention barrel-file discipline and clear public APIs per module so refactors stay contained.

**Q How do you design a reusable component for a shared library?**

- **Composition over configuration** — expose sub-components and `children` rather than a giant props object (compound components, e.g. `<Select><Select.Option/></Select>`).
- Sensible defaults, but allow overrides — `className` passthrough, `as`/polymorphic props, forwarded refs.
- **Controlled + uncontrolled** support so consumers choose.
- Accessibility baked in (roles, keyboard, focus management), strong TypeScript types, and no hidden coupling to app-specific state.

**Q What is an Error Boundary and where do you place them?**

A class component (or a library like `react-error-boundary`) implementing `getDerivedStateFromError`/`componentDidCatch` that **catches render-phase errors in its subtree** and shows a fallback instead of unmounting the whole app. Place them at meaningful boundaries — per route, per major widget — so one failing panel doesn't white-screen the page. Note the limits: they don't catch errors in event handlers, async code, or SSR.

**Q How do you version and roll out changes to a shared component library?**

**Semantic versioning** with a disciplined definition of a breaking change (prop removal/rename, behaviour change). Ship a changelog, deprecate before removing, and provide codemods for large migrations. Document components in **Storybook** with visual/interaction tests to catch regressions. In a monorepo, tools like Changesets manage versioning; consumers upgrade deliberately rather than being force-broken.

## 8 SPFx & SharePoint Online

*The differentiator for this specific role. Even "some experience" answered confidently stands out — be accurate about what you know.*

**Q What is the SharePoint Framework (SPFx) and why use it?**

SPFx is Microsoft's **client-side development model** for extending SharePoint Online and Microsoft 365. Your code runs **in the context of the page and the current user** (no iframes, browser-side), so components load fast and honour the user's permissions automatically. It's toolchain-based (Node, Yeoman generator, gulp/webpack) and supports React out of the box — Microsoft's own modern SharePoint UI is built with React and Fluent UI.

**Q What's the difference between a Web Part and an Extension?**

- **Client-Side Web Parts** — the main building block: reusable components users add and configure on a page (with a property pane for settings).
- **Extensions** — customise the SharePoint experience itself: *Application Customizers* (inject header/footer on every page), *Field Customizers* (render a list column), *Command Sets* (add list toolbar actions).

**Q Walk through the SPFx web part lifecycle.**

`onInit()` runs setup (get context, auth, service clients) before first render and returns a Promise. `render()` draws into the web part's DOM element — in a React web part you `ReactDOM.render` your root component here. The **property pane** methods (`getPropertyPaneConfiguration`, `onPropertyPaneFieldChanged`) manage configurable settings. `onDispose()` cleans up. Key point: unlike a standalone SPA, the web part is a component hosted inside SharePoint's page lifecycle.

**Q How do you call SharePoint / Microsoft Graph data from SPFx?**

Use the context-provided HTTP clients: **SPHttpClient** for SharePoint REST, **MSGraphClientV3** for Graph — both handle auth tokens for you. In practice most teams use **PnPjs**, a fluent wrapper that makes list/item/Graph operations far cleaner (e.g. `sp.web.lists.getByTitle('X').items()`). Permissions to Graph/APIs are granted via the tenant's **API access** page (Azure AD app permissions).

> **If pressed on depth:** Be honest about your level ("I've worked with SPFx web parts and PnPjs for list data; I'd ramp quickly on the areas I haven't touched"). Confident + accurate beats overclaiming — SharePoint devs spot bluffing instantly.

**Q How does SPFx handle permissions and security compared to a standalone SPA?**

SPFx runs as the **signed-in user**, so it inherits their SharePoint permissions — you don't manage a separate auth flow for SharePoint data. For external/Graph APIs you request scopes that a tenant admin approves centrally. Security responsibilities that remain yours: don't expose secrets in client code, sanitise any HTML you inject, respect least-privilege on requested scopes, and validate that the UI degrades gracefully when a user lacks access to an item.

**Q How do you bundle, deploy, and update an SPFx solution?**

Build produces a **.sppkg package** uploaded to the tenant **App Catalog**; assets are hosted (Office 365 CDN / SharePoint library). Admins deploy/approve it, then it's available to add to sites. Updates ship a new package version. Mention that you version carefully and test in a dev/test site collection first — matching the "development, test, and acceptance environments" the listing describes.

## 9 Accessibility & Frontend Quality

*The listing names accessibility principles explicitly — a quick, credible answer covers you.*

**Q How do you build accessible React components?**

- Use **semantic HTML first** (`<button>`, `<nav>`, `<label>`) — it gives roles and keyboard behaviour for free; reach for ARIA only to fill gaps.
- **Keyboard operability** — everything interactive reachable and usable via keyboard, visible focus states, logical tab order, focus management in modals.
- Label form fields, provide `alt` text, maintain colour contrast, and announce dynamic changes with `aria-live` regions.
- Test with **jest-axe** in unit tests and a screen reader / Lighthouse for real checks.

**Q What frontend security issues do you keep in mind?**

**XSS** is the big one — React escapes text by default, but `dangerouslySetInnerHTML` reopens the door, so sanitise (e.g. DOMPurify) any HTML you must inject. Also: never trust client-side validation alone, keep secrets/tokens out of the bundle, be careful with `target="_blank"` (`rel="noopener"`), and respect the app's Content Security Policy. In SharePoint specifically, honour tenant security and least-privilege API scopes.

**Q** **How would you explain a technical trade-off to a non-technical stakeholder?** (they'll ask — it's in the profile)

Frame it in **cost, risk, and outcome**, not jargon. Example: "Option A ships in a week but will be harder to change later; Option B takes three weeks but supports the features we know are coming next quarter." Give a recommendation with a reason, offer the decision, and confirm understanding. Interviewers assess whether you can be trusted in front of business analysts and clients — clarity and framing matter as much as the technical content.